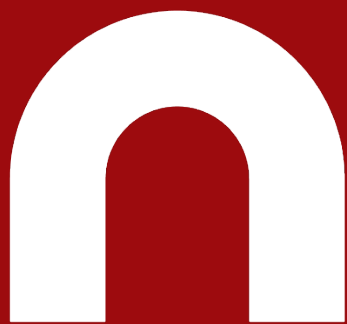




isec

Engenharia

DEPARTAMENTO DE INFORMÁTICA E SISTEMAS

Simulador de um jardim

Disciplina de Programação Orientada a Objetos

Autores

Orlando Nogueira de Souza dos Santos — 2023134097

Samuel Brito Correia Olavo — 2022130286

Coimbra, dezembro 2025

RESUMO

O presente relatório descreve o desenvolvimento do trabalho prático da unidade curricular de Programação Orientada a Objetos, que consiste na implementação de um simulador de um jardim, executado em ambiente de consola e baseado numa grelha bidimensional. Neste simulador, um jardineiro interage com o jardim, utilizando ferramentas e executando ações que influenciam o estado do solo e o comportamento das plantas ao longo de um tempo simulado.

O principal objetivo do trabalho é aplicar, de forma correta e consistente, os conceitos fundamentais da programação orientada a objetos, nomeadamente a definição de classes, a utilização de herança e polimorfismo, o recurso a classes abstratas, a gestão dinâmica de memória e a separação clara de responsabilidades entre os diferentes componentes do sistema, respeitando simultaneamente as restrições e requisitos definidos no enunciado.

O desenvolvimento do projeto foi realizado em duas fases distintas. Na Meta 1, procedeu-se ao planeamento e à definição da arquitetura do sistema, com especial enfoque na identificação das classes principais e abstratas, nas relações de herança e polimorfismo e na validação sintática dos comandos introduzidos pelo utilizador. Na Meta 2, foi implementada a totalidade das funcionalidades do simulador, incluindo a simulação do tempo, o comportamento dinâmico das plantas, a utilização das ferramentas, a interação do jardineiro com o jardim e a execução efetiva dos comandos.

Ao longo deste relatório são apresentadas a arquitetura do sistema, a modelação das classes, as estruturas de dados utilizadas, as opções de implementação adotadas e o grau de cumprimento dos requisitos definidos no enunciado.

ÍNDICE	
RESUMO	1
ÍNDICE	2
INTRODUÇÃO	4
Conceitos Fundamentais Utilizados	5
2.1. Encapsulamento e Separação de Responsabilidades	5
2.2. Herança e Especialização de Classes	5
2.3. Polimorfismo	6
2.4. Classes Abstratas e Métodos Virtuais	6
2.5. Gestão de Memória Dinâmica	7
2.6. Utilização da Classe de Configuração (Settings)	7
Arquitetura do Sistema	8
3.1. Visão Geral do Simulador	8
3.2. Organização interna do controlador	8
Interface do Utilizador (UI)	8
Sistema de Comandos	8
Simulador	9
Jardim	9
Jardineiro	9
Função principal (main)	9
3.3. Organização do Código	11
	12
4. Modelação e estrutura das classes	12
4.1. Hierarquia de Plantas	12
Caracterização estética das plantas	13
Espécies de plantas implementadas	13
Planta exótica – Maracujazeiro	14
4.2. Hierarquia de Ferramentas	15
Utilização das Ferramentas	15
Ferramentas implementadas	15
Drone de irrigação	15
4.3. Sistemas de Comandos	16
Classe base abstrata Comandos	16
Polimorfismo na execução dos comandos	17
Validação dos comandos	17
4.4. Outras classes de Suporte	18
Classe Random	18
Classe Settings	18
5. Estruturas de Dados e Representação do Jardim	18
5.1. Representação do jardim	19
5.2. Classe Solo	19
5.3. Atualização do jardim ao longo do tempo	19
5.4. Gestão dinâmica de memória	20
5.5. Estado global da simulação	20
6. Decisões de Implementação e Alterações ao Enunciado	20
6.1. Ação de entrada e saída do jardineiro como consumo de turno	21

6.2. Introdução do comando <code>usa</code> para ferramentas	21
<i>Funcionalidades Implementadas e Grau de Cumprimento</i>	22
<i>Conclusão</i>	24

INTRODUÇÃO

O presente relatório descreve o desenvolvimento do trabalho prático da unidade curricular de Programação Orientada a Objetos, como já referido anteriormente, cujo objetivo é a implementação de um simulador de um jardim, executado em ambiente de consola e baseado numa grelha bidimensional. Neste simulador, o utilizador assume o papel de um jardineiro, podendo interagir com o jardim através da execução de comandos que permitem realizar ações sobre o solo, as plantas e as ferramentas disponíveis.

O desenvolvimento do projeto foi dividido em duas fases. Na **Meta 1**, procedeu-se ao planeamento e à estruturação inicial do sistema, com enfoque na identificação das classes fundamentais e abstratas, na definição das relações de herança e polimorfismo e na validação sintática dos comandos introduzidos pelo utilizador. Esta fase teve como objetivo estabelecer uma base sólida para a implementação do simulador. Na Meta 2, foi realizada a implementação completa do sistema, integrando a simulação do tempo, o comportamento dinâmico das plantas, a utilização das ferramentas, a interação do jardineiro com o jardim e a execução efetiva dos comandos.

Durante a implementação foram introduzidas adaptações pontuais ao enunciado original, devidamente documentadas ao longo do relatório. Em particular, foi criado o comando `usa`, que não se encontrava explicitamente previsto no enunciado, com o objetivo de tornar a utilização das ferramentas mais controlada e previsível. Ao contrário do comportamento automático inicialmente descrito, a ação das ferramentas ocorre apenas quando o utilizador executa explicitamente o comando `usa`, sendo obrigatória a indicação do identificador da ferramenta a utilizar. Esta opção permite evitar efeitos não intencionais e facilita o controlo da simulação por parte do utilizador.

Ao longo deste relatório são apresentadas a arquitetura do sistema, a modelação das classes, as estruturas de dados utilizadas, as opções de implementação adotadas e o grau de cumprimento dos requisitos definidos no enunciado.

Conceitos Fundamentais Utilizados

De forma a facilitar a compreensão do trabalho desenvolvido, nesta secção são revistos os principais conceitos de Programação Orientada a Objetos abordados ao longo do semestre e aplicados na implementação do simulador. Estes conceitos foram fundamentais para a estruturação do sistema, permitindo uma organização modular, extensível e coerente com o domínio do problema.

2.1. Encapsulamento e Separação de Responsabilidades

O encapsulamento é um dos princípios base da programação orientada a objetos e foi aplicado ao longo do projeto. Cada classe foi concebida para representar uma entidade concreta do simulador, sendo responsável apenas pelo comportamento e pelos dados que lhe dizem diretamente respeito.

Neste sistema, a classe `Simulador` assume o papel de coordenador geral, sendo responsável pela interação com o utilizador, pela gestão do ciclo da simulação e pela articulação entre os diferentes componentes. A classe `Jardim`, por sua vez, é responsável pela gestão do espaço de simulação, incluindo a grelha do jardim, o controlo dos recursos disponíveis, o estado do solo e o comportamento das plantas ao longo do tempo.

A classe `Jardineiro` concentra a lógica comportamental associada às ações que podem ser realizadas sobre o jardim, como plantar, colher, mover-se e utilizar ferramentas, entre outras. O jardineiro não gere diretamente a interação com o utilizador nem o ciclo global da simulação, limitando-se a executar ações concretas quando solicitado.

Por fim, a classe `Solo` representa cada célula individual do jardim, armazenando informação relativa à quantidade de água e nutrientes presentes, bem como referências para os elementos que nela se encontram.

2.2. Herança e Especialização de Classes

A herança foi utilizada para modelar relações naturais entre entidades do sistema que partilham características comuns, mas que apresentam comportamentos distintos. Este conceito foi aplicado de forma transversal, nomeadamente nas hierarquias de plantas, ferramentas e comandos.

A classe abstrata `Planta` define as características e comportamentos comuns a todas as plantas, como a existência de reservas internas de água e nutrientes e a reação à passagem do tempo. As diferentes espécies de plantas, como *cactos*, *roseiras*, *ervas daninhas* e a *planta exótica*, são implementadas como classes derivadas, especializando o comportamento definido na classe base.

De forma semelhante, a classe abstrata `Ferramenta` define a interface comum a todas as ferramentas, sendo posteriormente especializada por classes derivadas como o *regador*, o *pacote de adubo*, a *tesoura de poda* e o *drone*. Esta abordagem permite adicionar novas ferramentas ao sistema sem necessidade de alterar a lógica existente.

2.3. Polimorfismo

O polimorfismo foi aplicado para permitir que diferentes objetos, pertencentes a classes derivadas distintas, possam ser tratados de forma uniforme através de referências ou ponteiros para classes base.

No simulador, este conceito é particularmente visível no tratamento das plantas e das ferramentas. Apesar de cada planta apresentar comportamentos específicos, todas respondem de forma polimórfica à passagem do tempo, sendo o método correspondente invocado de acordo com o tipo concreto da planta presente em cada posição do jardim.

O mesmo se verifica na execução dos comandos introduzidos pelo utilizador. Cada comando é implementado como uma classe derivada de uma classe abstrata comum, sendo executado através de um método virtual, o que permite uma execução transparente e extensível dos diferentes tipos de comandos.

2.4. Classes Abstratas e Métodos Virtuais

O uso de classes abstratas foi essencial para definir contratos comuns entre grupos de classes relacionadas. As classes abstratas não representam entidades derivadas do simulador, mas sim conceitos gerais que servem de base para especializações.

No projeto, destacam-se as classes abstratas `Planta`, `Ferramenta` e `Comando`. Estas classes definem métodos virtuais que são obrigatoriamente implementados pelas classes derivadas, garantindo que todas seguem uma interface comum.

Esta abordagem promove a reutilização de código, facilita a extensibilidade do sistema e reforça a aplicação correta do conceito de polimorfismo, uma vez que o comportamento concreto é decidido em tempo de execução.

2.5. Gestão de Memória Dinâmica

A gestão dinâmica de memória foi um aspeto central na implementação do simulador, especialmente devido às restrições impostas pelo enunciado. A grelha do jardim é representada através de uma estrutura retangular alocada dinamicamente, evitando o uso de coleções da biblioteca standard.

Para além disso, plantas e ferramentas são geridas através de ponteiros, permitindo a criação e destruição dinâmica destes objetos ao longo da simulação. Foram tomadas precauções para garantir a libertação correta da memória associada aos objetos, evitando fugas de memória e assegurando a estabilidade do sistema.

2.6. Utilização da Classe de Configuração (Settings)

Todos os valores numéricos relevantes do simulador, como quantidades iniciais de água e nutrientes, limites de absorção, condições de morte e parâmetros de multiplicação das plantas, são obtidos a partir da classe `Settings`, conforme exigido pelo enunciado.

Esta abordagem permite centralizar a configuração do sistema, facilitando a realização de testes e a adaptação do comportamento do simulador sem necessidade de alterar o código principal. Para efeitos de desenvolvimento, os valores das constantes podem ser ajustados, sendo posteriormente repostos na versão final entregue.

Arquitetura do Sistema

Neste capítulo é descrita a arquitetura geral do sistema, identificando os principais componentes do simulador, as suas responsabilidades e a forma como interagem entre si, permitindo compreender a organização global do projeto antes da apresentação detalhada das classes.

3.1. Visão Geral do Simulador

O sistema desenvolvido segue uma arquitetura modular, orientada a objetos, onde cada componente representa uma entidade concreta do domínio do problema. O simulador assenta numa grelha bidimensional que representa o jardim, sendo a interação com o utilizador realizada através de comandos textuais introduzidos na consola.

A arquitetura foi concebida de forma a separar claramente a interação com o utilizador, a lógica de controlo da simulação e a representação interna do estado do jardim, permitindo um sistema organizado, extensível e de fácil manutenção. A evolução do sistema ao longo do tempo é controlada através de um modelo de tempo simulado (os turnos), independente do tempo real.

3.2. Organização interna do controlador

O simulador é constituído por vários componentes principais, organizados de forma a separar a interação com o utilizador, a coordenação da simulação, a execução das ações e a gestão do estado do jardim. Cada componente desempenha um papel específico no funcionamento global do sistema.

Interface do Utilizador (UI)

A Interface do Utilizador é responsável pela interação direta com o utilizador, permitindo a introdução de comandos e a apresentação da informação produzida pela simulação. Esta componente funciona como ponto de entrada do sistema, encaminhando os comandos introduzidos para processamento.

Sistema de Comandos

O sistema de comandos é responsável por interpretar, validar e executar as instruções introduzidas pelo utilizador. Cada comando corresponde a uma ação específica e apenas comandos válidos são encaminhados para execução no simulador.

Simulador

O simulador constitui o núcleo central do sistema, sendo responsável pela coordenação geral da simulação. Este componente gere o ciclo de execução, o tempo simulado e a articulação entre os restantes componentes, garantindo o funcionamento consistente do sistema.

Jardim

O jardim representa o espaço de simulação propriamente dito, modelado como uma grelha bidimensional. Este componente é responsável pela gestão do estado do jardim, incluindo o solo, as plantas e os recursos disponíveis ao longo do tempo.

Jardineiro

O jardineiro representa a entidade controlada pelo utilizador, responsável pela execução das ações que afetam o estado do jardim, de acordo com os comandos introduzidos e validados pelo sistema.

Função principal (main)

A função principal (`main`) tem como única responsabilidade a inicialização do sistema. Nesta função são criadas as instâncias principais do simulador e da interface do utilizador, sendo posteriormente transferido o controlo da execução para a interface.

Após a inicialização, todo o fluxo de interação com o utilizador e de execução da simulação é gerido pelas restantes componentes do sistema, não tendo a função `main` qualquer envolvimento adicional na lógica da aplicação.

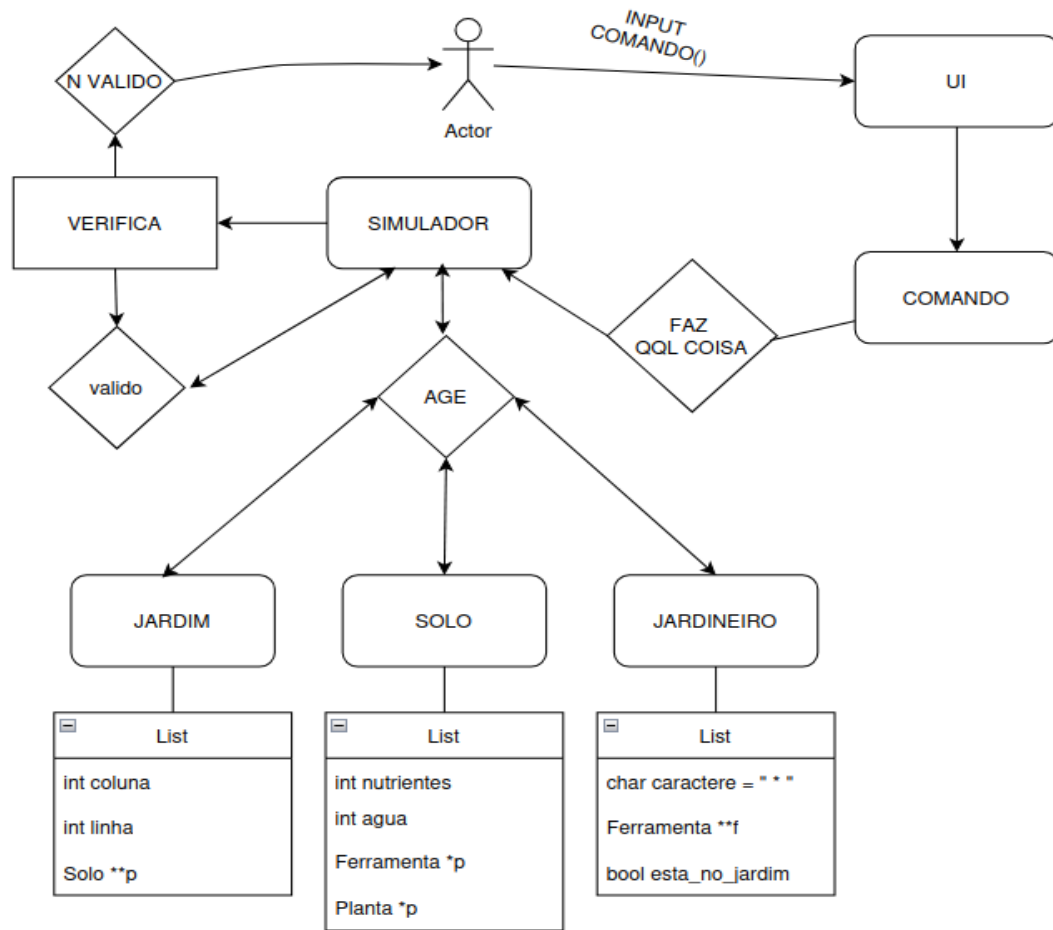


Figura 1-Exemplo de Comunicação Simulador com Classes

3.3. Organização do Código

O código do projeto encontra-se organizado de forma modular, refletindo a arquitetura do sistema e a separação de responsabilidades entre os diferentes componentes. As classes estão agrupadas por funcionalidade, facilitando a leitura do código, a sua manutenção e a evolução do simulador.

A tabela seguinte apresenta a estrutura do projeto e a responsabilidade principal de cada pasta e ficheiro.

Pasta/ficheiro	Descrição
Main.cpp	Ponto de entrada da aplicação; inicializa o simulador e a interface do utilizador.
InterfaceUtilizador/	Implementa a interface de interação com o utilizador, responsável pela leitura de comandos e apresentação de informação.
Simulador/	Contém a classe responsável pela coordenação geral da simulação e pelo controlo do tempo simulado.
Jardineiro/	Implementa a entidade controlada pelo utilizador e a lógica associada às ações executáveis.
Mapa/	Contém a classe <code>Jardim</code> , responsável pela gestão da grelha do jardim e do seu estado global.
Solo/	Implementa a classe <code>Solo</code> , que representa cada célula individual do jardim.
Planta/	Implementa a hierarquia de plantas, incluindo a classe base abstrata e as diferentes espécies.
Ferramentas/	Implementa a hierarquia de ferramentas disponíveis no simulador.
Comandos/	Contém o sistema de comandos, incluindo validação e execução.
Comandos/Acoes/	Comandos associados a ações que alteram o estado do jardim.
Comandos/Listar/	Comandos de listagem e consulta de informação.

Comandos/Movimento/	Comandos relacionados com o movimento do jardineiro.
Comandos/Gerais/	Comandos de carácter geral, como gravação, recuperação e execução de ficheiros.
Comandos/CriarMapa/	Comando responsável pela criação inicial do jardim.
Comandos/Fim/	Implementa o comando de encerramento da simulação.
Random/	Funções auxiliares para geração de valores aleatórios.
Settings/	Contém a classe de configuração com os parâmetros globais do simulador.

Tabela 1- Organização do código

4. Modelação e estrutura das classes

Neste capítulo é apresentada a modelação do sistema desenvolvido, com foco na estrutura de classes que suporta o simulador. São identificadas as classes fundamentais e as principais hierarquias utilizadas, bem como as relações existentes entre elas, de acordo com os princípios da Programação Orientada a Objetos.

A organização apresentada resulta do planeamento realizado na Meta 1 e da implementação efetuada na Meta 2, refletindo a evolução do projeto desde a fase conceptual até à concretização do simulador funcional.

4.1. Hierarquia de Plantas

As plantas constituem um dos elementos centrais do simulador, sendo responsáveis por grande parte da dinâmica do jardim ao longo do tempo simulado. Apesar de apresentarem comportamentos distintos consoante a espécie, todas partilham um conjunto de características comuns, o que motivou a utilização de uma hierarquia de classes baseada em herança.

Assim sendo, foi definida a classe base abstrata `Planta`, que representa o conceito genérico de planta no simulador. Esta classe concentra os atributos e comportamentos comuns a todas as espécies, nomeadamente a gestão das reservas internas de água e nutrientes, o estado de vida da planta e a sua caracterização estética. A classe `Planta`

define ainda a interface que todas as plantas derivadas são obrigadas a implementar, permitindo que o simulador trate diferentes espécies de forma uniforme.

Caracterização estética das plantas

Cada espécie de planta apresenta uma caracterização estética, representada por um valor de beleza. Este valor pode assumir um de três estados possíveis: **feia**, **neutra** ou **bonita**, sendo uma propriedade intrínseca da espécie.

A caracterização estética não influencia diretamente o comportamento funcional das plantas nem a evolução da simulação ao longo do tempo. O seu principal objetivo é permitir a distinção visual e descritiva das diferentes espécies, nomeadamente nos comandos de listagem e apresentação de informação ao utilizador.

Espécies de plantas implementadas

A partir da classe base `Planta` foram implementadas várias classes derivadas, cada uma representando uma espécie com comportamento próprio, conforme definido no enunciado:

`Cacto` – Planta resistente, com baixo consumo de água, adequada a ambientes com poucos recursos, associada a uma beleza neutra.

`Roseira` – Planta com necessidades equilibradas de água e nutrientes, associada a uma beleza bonita, representando uma espécie mais exigente, mas esteticamente valorizada.

`Erva-daninha` – Planta de crescimento rápido e comportamento invasivo, associada a uma beleza feia, podendo multiplicar-se facilmente e competir com outras plantas pelos recursos do solo.

`Maracujazeiro` – Planta exótica definida no âmbito do trabalho prático, com características próprias e comportamento distinto das restantes espécies.

Planta exótica – Maracujazeiro

A planta exótica implementada no simulador corresponde a um maracujazeiro, uma planta trepadeira produtiva, de beleza considerada bonita, com elevadas exigências de água e nutrientes. O seu comportamento foi definido de forma a representar uma planta mais complexa, conforme permitido pelo enunciado, recompensando uma gestão cuidada dos recursos do solo.

O maracujazeiro apresenta um consumo significativo de água e nutrientes a cada instante, bem como perdas regulares associadas ao seu crescimento rápido. Quando bem mantido, pode atingir valores elevados de reservas internas, permitindo a sua multiplicação para posições vizinhas vazias do jardim. Por outro lado, situações extremas, como falta prolongada de água, ausência de nutrientes ou excesso de água durante vários instantes consecutivos, conduzem à morte da planta.

Quando ocorre a morte, parte dos recursos acumulados pelo maracujazeiro é devolvida ao solo, contribuindo para a dinâmica global do ecossistema do jardim. Este comportamento torna a planta exótica um elemento particularmente relevante na simulação, exigindo ao utilizador uma gestão equilibrada e estratégica.

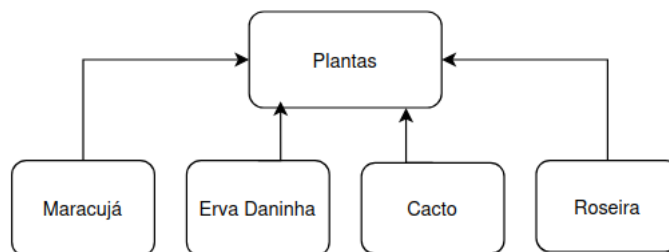


Figura 2-xemplo de Polimorfismo aplicado nas plantas

4.2. Hierarquia de Ferramentas

As ferramentas constituem um dos meios através dos quais o utilizador pode intervir ativamente no estado do jardim, permitindo modificar as condições do solo e influenciar o comportamento das plantas ao longo do tempo simulado. Tal como definido no enunciado, as ferramentas apresentam comportamentos distintos, mas partilham um conjunto de características comuns, o que motivou a utilização de uma hierarquia de classes baseada em herança.

Para esse efeito, foi definida a classe base abstrata `Ferramenta`, que representa o conceito genérico de ferramenta no simulador. Esta classe estabelece a interface comum a todas as ferramentas, garantindo que todas podem ser utilizadas de forma uniforme pelo sistema, independentemente do seu comportamento concreto.

Utilização das Ferramentas

A utilização de **qualquer ferramenta** é realizada de forma explícita através do comando `usa`. Este comando recebe como parâmetro o identificador da ferramenta a utilizar, garantindo que apenas ações intencionalmente solicitadas pelo utilizador afetam o estado do jardim. O enunciado do trabalho não abordava este comando, contudo iremos ainda abordar este comando mais à frente no relatório e explicar o porquê da sua implementação.

Ferramentas implementadas

A partir da classe base `Ferramenta` foram implementadas várias classes derivadas, correspondentes às ferramentas previstas no enunciado, bem como uma ferramenta adicional definida no âmbito do trabalho prático:

Regador – Ferramenta utilizada para aumentar a quantidade de água no solo, permitindo ao utilizador compensar a falta de água em determinadas posições do jardim.

Pacote de adubo – Ferramenta responsável por aumentar a quantidade de nutrientes disponíveis no solo, favorecendo o crescimento das plantas.

Tesoura de poda – Ferramenta utilizada para remover plantas do jardim, permitindo ao utilizador controlar a ocupação do espaço e eliminar plantas indesejadas.

Drone de irrigação – Ferramenta adicional definida no âmbito do trabalho prático, com comportamento distinto das restantes ferramentas.

Drone de irrigação

Como ferramenta adicional (`Ferramenta Z`), foi implementado um **drone de irrigação**, concebido para atuar sobre múltiplas posições do jardim de forma automática. O drone

permite ao utilizador seleccionar **uma linha ou uma coluna** do jardim, irrigando todas as posições dessa linha ou coluna numa única utilização.

À semelhança das restantes ferramentas, o drone é utilizado através do comando `usa`. No entanto, distingue-se pelo seu impacto mais abrangente e pelo facto de possuir **apenas uma utilização**, sendo descartado após a sua execução.

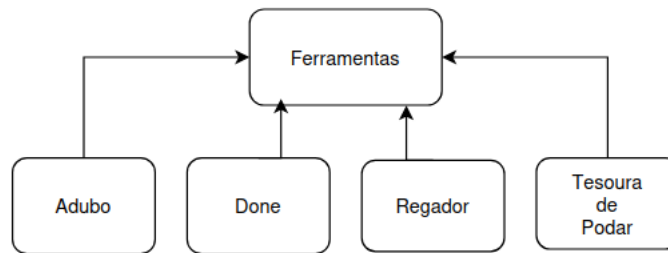


Figura 3- Exemplo de Polimorfismo aplicado nas Ferramentas

4.3. Sistemas de Comandos

A interação entre o utilizador e o simulador é realizada através de um sistema de comandos introduzidos na consola. Este sistema foi concebido, recorrendo a herança, polimorfismo e encapsulamento para garantir uma estrutura escalável e de fácil manutenção.

Cada comando corresponde a uma ação específica e é implementado como uma classe concreta, derivada de uma classe base abstrata comum.

Classe base abstrata Comandos

A classe abstrata `Comandos` define a interface comum a todos os comandos disponíveis no simulador. Esta classe contém informação partilhada por todos os comandos, nomeadamente a frase introduzida pelo utilizador e uma referência ao simulador, permitindo que cada comando interaja diretamente com o estado da simulação.

A classe `Comandos` define ainda o método virtual puro `executa()`, que é obrigatoriamente implementado por todas as classes derivadas, garantindo que cada comando possui uma ação concreta associada. Esta abordagem permite tratar todos os comandos de forma uniforme, independentemente do seu tipo específico.

Polimorfismo na execução dos comandos

O sistema recorre a polimorfismo para a execução dos comandos. Após a criação de um comando concreto, este é tratado através de um ponteiro para a classe base `Comandos`, sendo a execução realizada através de uma chamada polimórfica ao método `executa()`.

Desta forma, o simulador não necessita de conhecer o tipo concreto do comando a executar, limitando-se a invocar a interface definida pela classe base. O comportamento específico é determinado dinamicamente em tempo de execução, de acordo com a classe concreta do comando.

Validação dos comandos

A validação dos comandos é realizada de forma centralizada na classe base `Comandos`. Métodos auxiliares permitem verificar o número de argumentos fornecidos pelo utilizador e garantir que a utilização do comando respeita o formato esperado.

Este mecanismo evita duplicação de código nas classes derivadas e contribui para a robustez do sistema, assegurando que apenas comandos válidos são executados.

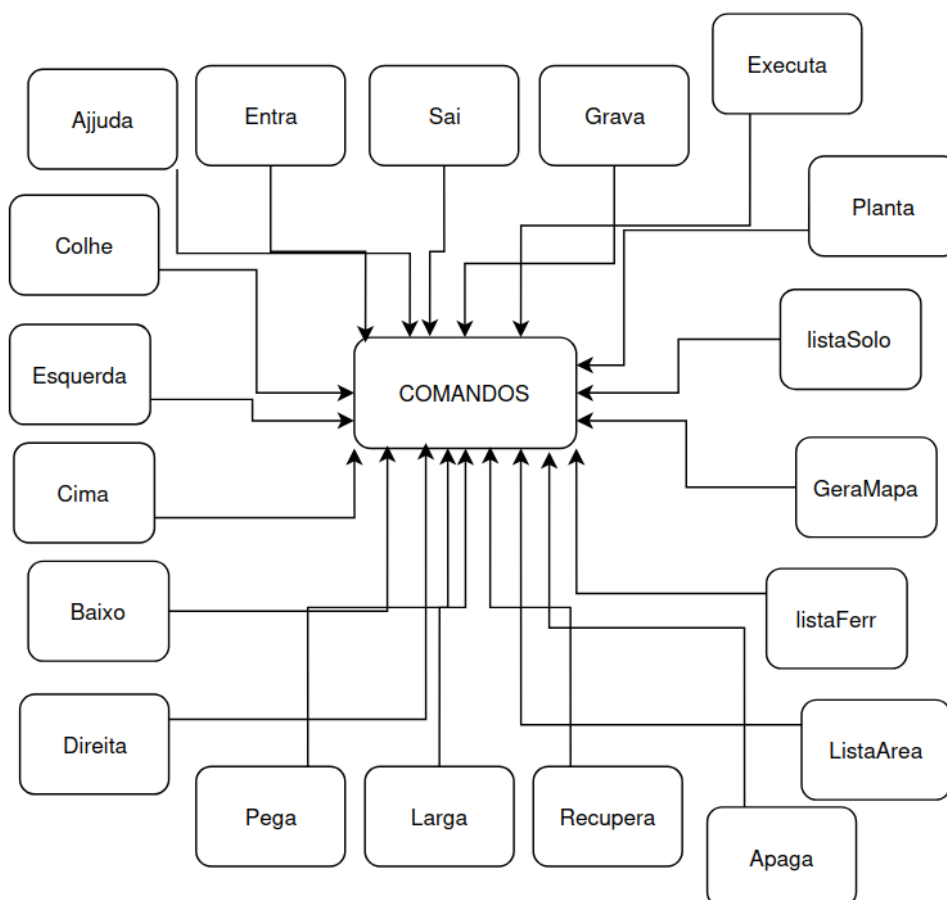


Figura 4- Exemplo de Polimorfismo aplicado nos Comandos

4.4. Outras classes de Suporte

Para além das classes principais que constituem o núcleo da simulação, o sistema recorre a um conjunto de **classes de suporte**, cujo objetivo é auxiliar o funcionamento global da aplicação, promovendo uma maior organização, reutilização de código e facilidade de manutenção.

Estas classes não representam entidades do domínio do problema, mas desempenham um papel fundamental no apoio à simulação e à interação com o utilizador.

Classe `Random`

A classe `Random` é responsável por gerar valores aleatórios utilizados em diferentes momentos da simulação, nomeadamente na colocação inicial de elementos no jardim e em comportamentos que dependem de fatores aleatórios.

A centralização da geração de números aleatórios nesta classe evita a dispersão de chamadas diretas a funções de geração aleatória ao longo do código, garantindo maior controlo, consistência e clareza na implementação.

Classe `Settings`

A classe `Settings` é utilizada para agrupar parâmetros globais do simulador, como valores por defeito e constantes utilizadas na simulação. Esta abordagem evita a utilização de valores codificados diretamente no código, facilitando a manutenção e a alteração dos parâmetros do sistema.

A utilização desta classe contribui para uma melhor separação entre lógica de funcionamento e configuração do simulador.

5. Estruturas de Dados e Representação do Jardim

Neste capítulo são descritas as estruturas de dados utilizadas para representar o jardim e os seus elementos, bem como as decisões de implementação relacionadas com a organização espacial e a gestão dinâmica de memória. A modelação adotada visa garantir uma separação clara de responsabilidades entre a gestão global do espaço e o estado local de cada posição do jardim.

5.1. Representação do jardim

O jardim é representado através de uma **grelha bidimensional**, cujas dimensões são definidas em tempo de execução pelo utilizador. Esta grelha é implementada como uma matriz dinâmica de objetos da classe `Solo`, permitindo uma representação flexível do espaço de simulação.

A grelha é gerida pela classe `Jardim`, que é responsável pela sua criação, inicialização e destruição. Cada posição da grelha corresponde a uma instância independente de `Solo`, permitindo uma gestão local do estado do jardim.

Esta abordagem facilita a navegação do jardineiro, a localização de plantas e ferramentas, e a implementação de ações que dependem da posição no espaço.

5.2. Classe `Solo`

A classe `Solo` representa uma célula individual da grelha do jardim e constitui a unidade fundamental da simulação. Cada instância de `Solo` armazena o estado local de uma posição específica, incluindo a quantidade de água e nutrientes disponíveis, bem como a eventual presença de uma planta, de uma ferramenta e do jardineiro.

A separação entre a classe `Jardim`, responsável pela gestão global da grelha, e a classe `Solo`, responsável pela gestão local de cada posição, permite uma organização modular do sistema, facilitando a manutenção e a extensão do código.

O estado visual de cada posição do jardim é representado por um carácter, atualizado de acordo com os elementos presentes na célula, permitindo uma representação simples e eficaz do jardim em modo de consola.

5.3. Atualização do jardim ao longo do tempo

A evolução do estado do jardim ao longo do tempo é coordenada pela classe `Simulador`, que gere o conceito de **turnos de simulação**. O avanço do tempo é realizado de forma discreta, sendo cada turno responsável por atualizar o estado global da simulação.

Em cada avanço de turno, o simulador:

incrementa o contador de turnos;

repõe os limites de ações do jardineiro;

solicita ao jardim a atualização do estado das plantas em todas as posições da grelha.

A atualização das plantas é realizada percorrendo a totalidade da grelha e invocando o comportamento específico de cada planta presente. Caso uma planta deixe de estar viva após esta atualização, esta é removida do solo correspondente, sendo libertada a memória associada e atualizado o estado visual da célula.

5.4. Gestão dinâmica de memória

A simulação recorre extensivamente à **alocação dinâmica de memória**, permitindo a criação de jardins de dimensão variável e a gestão dinâmica de plantas, ferramentas e do jardineiro.

A classe `Jardim` é responsável pela alocação e destruição da grelha bidimensional, assegurando a libertação correta da memória no final da simulação. Cada instância de `Solo` é responsável pela destruição dos objetos que contém, nomeadamente plantas e ferramentas, garantindo uma gestão de memória consistente.

Adicionalmente, a utilização de destrutores virtuais nas classes base permite a destruição correta de objetos derivados, evitando fugas de memória e comportamentos indefinidos.

5.5. Estado global da simulação

O estado global da simulação é mantido na classe `Simulador`, que armazena informação relativa ao número de turnos decorridos, limites de ações por turno e referências para o jardim e o jardineiro.

O simulador suporta ainda a **gravação e recuperação do estado da simulação**, recorrendo à criação de cópias do jardim e do jardineiro. Estas cópias permitem restaurar um estado anterior da simulação, preservando a organização e o estado interno dos elementos.

Esta funcionalidade reforça a robustez do sistema e demonstra uma gestão cuidada das estruturas de dados e da memória.

6. Decisões de Implementação e Alterações ao Enunciado

Durante o desenvolvimento do simulador, foram tomadas algumas decisões de implementação que implicaram **ligeiros ajustamentos face ao enunciado original**. Estas alterações não descaracterizam o funcionamento geral do sistema, tendo sido introduzidas com o objetivo de melhorar a simulação, a clareza da interação com o utilizador e o controlo das ações ao longo do tempo simulado.

6.1. Ação de entrada e saída do jardineiro como consumo de turno

O simulador é estruturado em **turnos**, sendo que em cada turno o jardineiro dispõe de um número limitado de ações. Nesse contexto, foi decidido considerar a **entrada (teleporte) e saída do jardineiro do jardim como ações que consomem movimentos**.

Esta decisão foi tomada para evitar situações em que o utilizador pudesse teletransportar o jardineiro sem qualquer custo temporal, o que comprometeria o equilíbrio da simulação e a progressão dos turnos.

Assim, a entrada e saída do jardineiro são contabilizadas como ações dentro do turno, estando sujeitas a restrições temporais (nomeadamente limitações por número de turnos), promovendo uma utilização mais coerente do sistema.

6.2. Introdução do comando `usa` para ferramentas

De acordo com o enunciado, o jardineiro pode transportar várias ferramentas, estando apenas uma ativa (na mão), e a ação dessa ferramenta seria aplicada **automaticamente a cada instante**, enquanto se mantivesse ativa.

No entanto, durante a implementação, foi considerada mais adequada a introdução de um **comando `usa`** `<id>`, responsável por ativar a funcionalidade da ferramenta escolhida.

Esta alteração foi motivada pelas seguintes razões:

- Permitir ao utilizador **controlar explicitamente quando uma ferramenta é utilizada**, evitando efeitos automáticos indesejados a cada avanço de turno;
- Facilitar a alternância entre diferentes ferramentas, sem obrigar a que a ferramenta ativa execute sempre a sua ação;
- Tornar a interação mais clara e previsível do ponto de vista do utilizador, associando cada ação a um comando concreto;
- Uniformizar o comportamento de todas as ferramentas, garantindo que **todas são utilizadas através do mesmo mecanismo**, independentemente do tipo.

Desta forma, embora a lógica seja diferente da abordagem descrita no enunciado, o comando `usa` mantém a funcionalidade do sistema e melhora a experiência de utilização, sem alterar os princípios gerais definidos.

Importa salientar que **todas as ferramentas do simulador são utilizadas exclusivamente através do comando `usa`**, e não apenas o drone, garantindo consistência na interação.

Funcionalidades Implementadas e Grau de Cumprimento

A tabela seguinte resume as principais funcionalidades definidas no enunciado do trabalho prático, bem como o respetivo grau de cumprimento na implementação desenvolvida.

Requisito / Funcionalidade	Estado
Estrutura geral do sistema	
Simulador executado em ambiente de consola	Cumprido
Representação do jardim através de grelha bidimensional	Cumprido
Separação do sistema em múltiplas classes	Cumprido
Utilização de herança e polimorfismo	Cumprido
Utilização de classes abstratas	Cumprido
Gestão dinâmica de memória	Cumprido
Plantas	
Existência de uma classe base abstrata Planta	Cumprido
Implementação de várias espécies de plantas	Cumprido
Comportamento distinto por espécie	Cumprido
Implementação do Cacto	Cumprido
Implementação da Roseira	Cumprido
Implementação da Erva-daninha	Cumprido
Implementação de uma planta exótica	Cumprido
Planta exótica com regras próprias	Cumprido
Morte das plantas segundo condições definidas	Cumprido
Devolução de recursos ao solo aquando da morte	Cumprido
Multiplicação de plantas segundo regras próprias	Cumprido
Jardineiro	
Existência de um jardineiro no simulador	Cumprido
Entrada do jardineiro no jardim	Cumprido
Saída do jardineiro do jardim	Cumprido
Movimento do jardineiro na grelha	Cumprido
Limitação de ações por turno	Cumprido
Ferramentas	
Existência de uma classe abstrata Ferramenta	Cumprido
Implementação de ferramentas concretas	Cumprido
Ferramentas associadas a posições do solo	Cumprido
Ferramentas transportadas pelo jardineiro	Cumprido
Ação das ferramentas sobre o solo	Cumprido
Comandos e interação	
Interação exclusiva através de comandos textuais	Cumprido
Validação dos argumentos dos comandos	Cumprido
Implementação do comando <code>jardim</code>	Cumprido
Implementação do comando <code>avanca</code>	Cumprido
Implementação do comando <code>entra</code>	Cumprido
Implementação do comando <code>sair</code>	Cumprido
Implementação do comando <code>planta</code>	Cumprido
Implementação do comando <code>colhe</code>	Cumprido
Implementação do comando <code>lsolo</code>	Cumprido
Implementação do comando <code>lplanta</code>	Cumprido

Implementação do comando <code>lplantas</code>	Cumprido
Implementação do comando <code>larea</code>	Cumprido
Implementação do comando <code>fim</code>	Cumprido
Plantas	
Gravação do estado da simulação	Cumprido
Recuperação de estados guardados	Cumprido
Eliminação de estados guardados	Cumprido

Tabela 2- Funcionalidades e grau de cumprimento do que foi solicitado

Conclusão

A realização deste trabalho prático permitiu consolidar os conteúdos abordados ao longo das aulas teóricas e práticas de Programação Orientada a Objetos. A implementação do simulador de jardim foi fundamental para aplicar conceitos como encapsulamento, herança, polimorfismo, classes abstratas e gestão dinâmica de memória, num contexto prático e realista.

Ao longo do desenvolvimento surgiram vários desafios, nomeadamente na modelação do comportamento das plantas, na gestão do tempo simulado e na coordenação das ações do jardineiro. Estes desafios foram ultrapassados através de uma estruturação cuidada do código e de uma separação clara de responsabilidades entre as diferentes classes do sistema.

O resultado é um simulador funcional e alinhado com os objetivos definidos no enunciado do trabalho prático, permitindo uma interação clara com o utilizador e uma simulação dinâmica do jardim ao longo do tempo.